CS6650 Distributed Systems — Experiments Report

Flair2 — AI Campaign Studio

CS6650 Distributed Systems · Northeastern University · April 2026
Shuang (Sam) Wu & Jingsi (Jess) Zhang · github.com/yangyang-how/flair2

> **Flair2** is a six-stage distributed AI pipeline that analyzes viral TikTok videos, generates campaign scripts, evaluates them through simulated persona voting, and personalizes the top results to a creator's voice. The pipeline runs on **AWS ECS Fargate** with **ElastiCache Redis** as the shared state layer and **Celery** as the task queue. Every LLM call is IO-bound and expensive - the entire distributed design exists to run as many of them in parallel as the provider allows, recover cheaply when workers crash, and avoid redundant work when multiple users submit the same input.

Pipeline Architecture

```
S1 [Map]     → Analyze N=100 videos concurrently     → S1Pattern per video
S2 [Reduce]  → Aggregate patterns                    → S2PatternLibrary
S3           → Generate N=20 scripts (concurrent)     → CandidateScript[ ]
S4 [Map]     → N=42 personas vote (cap=29, Kimi)      → PersonaVote[ ]
S5 [Reduce]  → Borda-count ranking                    → top-N scripts
S6 [Map]     → Personalize top-N concurrently         → FinalResult[ ]
```

All Experiments

#	Experiment	Env	Tests	Key Result
1 ★	Fan-out / Fan-in Parallelism	Local	5/5	26× speedup at N=42, cap=29 (Kimi limit). Amdahl ceiling confirmed at cap=N.
2 ★	Straggler Mitigation (95% threshold)	Local	5/5	89.6% pipeline time saved under severe stragglers. <2% overhead without stragglers.
3 ★	API Concurrent Load Test (Locust)	AWS ECS	Live	K≤100 stable. K=500 inflection. Bottleneck: Redis connection pool, not Worker CPU.
M5-1	Backpressure (TokenBucket)	Local	6/6	Rate limiter: 0% errors vs 90% without at K=10. CV drops 1.36→0.40 (fairer at scale).
M5-2	Failure Recovery + Run Isolation	Local	5/5	47–73% API call savings from checkpoint on crash. Sibling runs unaffected.
M5-3	Cross-user Cache Concurrency (SETNX)	Local	6/6	90% LLM call savings at K=10. Scales to 99.999% at K=100,000.
M6	ElastiCache Integration (3 sub-exps.)	AWS Redis	27/29	Latency p50≈0.8ms. SETNX atomic ≤K=1000. Memory linear ~1KB/key.

★ = featured in depth (pages 2–4).
`Experiment code: backend/tests/experiments/`

EXPERIMENT 1

Fan-out Parallelism

Purpose	Tradeoff	Limitations
Quantify the speedup of running N independent IO-bound LLM tasks concurrently vs sequentially, and show how a concurrency cap bounds speedup.	More concurrency → faster pipeline, but the LLM provider enforces a hard cap (Kimi: 29 slots). Beyond the cap, more workers yield zero additional benefit.	Simulated with asyncio.sleep() as a proxy for LLM latency. Real Celery workers across ECS processes would show higher efficiency (no shared event loop/GIL).

Setup

We simulate S4's fan-out: N=42 personas each making one LLM call (100 ms latency). A semaphore of size *cap* models the `RedisSemaphore` that limits concurrent Kimi connections. We measure wall-clock completion time for *cap* ∈ {1 (serial), 5, 10, 29, ∞}. Theoretical speedup = min(N, cap); efficiency = actual speedup / theoretical.

Code: `tests/experiments/test_fanout_parallelism.py`

Results

Cap	Time (ms)	Theoretical ×	Actual ×	Efficiency
1 (serial)	4,200	1.0	1.0	100%
5	~900	5.0	4.7	93%
10	~450	10.0	9.3	93%
29 (Kimi prod)	~160	29.0	26.3	90.7%
∞ (uncapped)	~105	42.0	40.0	95%

Completion time by concurrency cap (N=42 personas, 100 ms per LLM call):

cap=01	4,200 ms	1× speedup
cap=05	~900 ms	4.7× speedup
cap=10	~450 ms	9.3× speedup
cap=29	~160 ms	26× speedup
cap=∞	~105 ms	40× speedup

Amdahl ceiling: at N=10, time difference between cap=10 and cap=100 is <1% — speedup plateaus at cap≥N.

Analysis

- **Fan-out achieves near-linear speedup for IO-bound tasks.**
  Without a concurrency cap, 42 LLM calls complete in ≈1 latency unit (105 ms) instead of 42 (4,200 ms). With Kimi's 29-slot cap, speedup is 26×. This directly justifies the fan-out architecture for S4: a sequential S4 would add over 4 seconds per pipeline run; the fan-out reduces this to ~160 ms.

- **Amdahl's Law holds cap=29 is the binding constraint, not ECS task count.**
  Adding more Celery workers beyond what saturates Kimi's 29 slots yields zero additional benefit. The right optimization is increasing the provider concurrency limit, not scaling the ECS worker service.

- **Efficiency drops slightly at high cap (90% at cap=29 vs 95% uncapped)**
  due to asyncio event-loop scheduling overhead. In production with real Celery workers across multiple ECS processes, efficiency would be higher as tasks run in parallel OS threads.

EXPERIMENT 2

Straggler Mitigation — 95% Completion Threshold

Purpose	Tradeoff	Limitations
Validate that the 95% threshold in Orchestrator._try_transition() reduces pipeline latency when a small fraction of tasks take disproportionately long.	Waiting for 100% ensures all results arrive before the next stage, but one straggler (e.g. 90 s rate-limit backoff) blocks 41 completed tasks. 95% threshold advances early — straggler results still arrive and are stored.	Stragglers simulated by inflating asyncio.sleep(). Real stragglers arise from LLM provider variance, network jitter, and rate-limit backoff — harder to control deterministically.

Setup

N=100 tasks (matching S1 production: 100 videos), base latency 50 ms. The slowest 5% (5 tasks) are inflated by a multiplier (3×, 10×, 20×). We measure the time at which the *next-stage trigger fires* under threshold=1.00 (wait for all) vs threshold=0.95 (fire at ⌈100×0.95⌉=95 completions). All tasks still complete; only the trigger timing changes.

For the S4 production test: N=42, 2 stragglers at 10× (⌈42×0.95⌉=40 needed; 40 fast tasks complete before any straggler).

Code: `tests/experiments/test_straggler_mitigation.py`

Results

Scenario	Straggler delay	t=100% (ms)	t=95% (ms)	Time saved
Mild	3× median (150 ms)	151	52	65.4%
Severe	**10× median (500 ms)**	**501**	**52**	**89.6%**
Worst case	20× median (1,000 ms)	1,002	52	94.8%
No stragglers	1× (uniform)	52	51	1.9%

S4 production config (N=42, 2 stragglers at 10×): 501 ms → 52 ms → **89.7% time saved**

Time saved by 95% threshold vs waiting for 100%:

Mild (3×)	65.4% saved	149 ms recovered
Severe (10×)	89.6% saved	449 ms recovered
Worst (20×)	94.8% saved	950 ms recovered
No stragglers		<2% overhead

Analysis

- **The 95% threshold eliminates straggler-induced pipeline delays with near-zero cost.**
  Under uniform latency (no stragglers), the overhead is <2 ms — negligible. Under severe stragglers (10× median, realistic for a LLM rate-limit backoff of 90s), the threshold saves 89.6% of pipeline waiting time. The mechanism is free in the common case and highly effective in the worst case.

- **This is equivalent to speculative execution in distributed systems.**
  Just as MapReduce launches speculative duplicate tasks to avoid waiting for slow nodes, Flair2's 95% threshold advances the pipeline without the late-completing tasks. The straggler tasks still complete and store their results — the pipeline simply doesn't wait for them.

- **Key invariant validated:** all tasks complete after the trigger fires (no cancellation). The straggler results are stored n Redis and available for future analysis, even if they arrived after the next stage began.

EXPERIMENT 3

API Concurrent Load Test (AWS ECS + ALB)

Purpose	Tradeoff	Limitations
Identify where the system breaks under concurrent user load and find the true scaling bottleneck — API, Worker, or shared state layer?	More concurrent users → higher throughput, but at some threshold Redis connections and ECS CPU become contended. The wrong scaling metric can fail to trigger auto-scaling in time.	Locust payload uses num_videos=2 to avoid overwhelming the LLM provider. This tests API/infrastructure throughput, not end-to-end pipeline latency with real LLM calls.

Setup

Locust simulates K concurrent virtual users against the deployed ALB (`flair2-dev-alb.us-west-2.elb.amazonaws.com`). Each user sends a realistic mix: `POST /api/pipeline/start` (weight 3), `GET /api/runs` (weight 2), `GET /api/health` (weight 1), `GET /api/providers` (weight 1). Wait time 1–3 s between tasks. Run three times: Day 1 (cumulative load), Day 2 (clean cluster baseline), Day 3 (post code changes).

Code: `tests/experiments/locustfile.py`, `run_load_test.sh`

Results

K	p50 (ms)	p95 (ms)	p99 (ms)	RPS	Failures
10	35	51	190	4.81	0%
50	35	59	100	48.24	0%
100	34	61	140	47.96	0%
500 (Day 1)	**69**	**2,900**	**4,200**	**175.89**	**0.03%**
500 (Day 3, clean)	420	1,300	2,900	87.1	0%

p95 latency by K — POST /api/pipeline/start:

K=10	■	51 ms
K=50	■	59 ms
K=100	■	61 ms
K=500 Day 1	■■■

Additional Experiments

The following experiments validate complementary resilience and infrastructure properties of the pipeline.

M5: Pipeline Resilience (fakeredis, 17/17 tests)

Experiment	What We Measured	Key Result
M5-1 Backpressure	Error rate when K concurrent users share one LLM rate limit. No-limiter vs TokenBucketRateLimiter.	90% errors without limiter at K=10. Rate limiter holds 0% across all K. Fairness CV: 1.36 → 0.40.
M5-2 Failure Recovery	API call savings when a worker crashes mid-S4 and resumes from Redis checkpoint. Crash at 25/50/75%.	47%/60%/73% API calls saved. Sibling runs complete with zero data contamination from crashed run.
M5-3 Cache Concurrency	Whether SETNX prevents duplicate LLM calls when K users submit identical videos.	SETNX calls always = NUM_VIDEOS regardless of K. 90% savings at K=10 → 99.999% at K=100,000.

M6: ElastiCache Integration (AWS real Redis, 27/29 tests)

Experiment	What We Measured	Key Result
M6-1 Latency	p50/p95/p99 latency for SETNX, XADD, INCR on real ElastiCache vs fakeredis.	p50 ≈ 0.8 ms, p99 < 2 ms. ~10× slower than fakeredis (0.05 ms). Within SLA.
M6-2 SETNX Atomicity	Whether SETNX guarantees exactly one winner under real concurrent load (K=10–5,000).	Exactly 1 winner at K=10–1,000. Fails at K=5,000: aioredis pool exhaustion — not Redis server fault.
M6-3 Memory	Whether memory scales linearly within cache.t3.micro limits as concurrent runs grow.	~1 KB/key, linear. cache.t3.micro safe to ~300 concurrent runs. 500+ needs larger instance.

Cross-Experiment Conclusions

- **Fan-out creates the straggler problem; the threshold solves it.** Dispatching 42 concurrent LLM tasks means completion time is determined by the slowest. Without the 95% threshold, one rate-limited persona vote blocks 41 completed ones. The two mechanisms are complementary: fan-out maximizes throughput, the threshold bounds variance.
- **The load test reveals the infrastructure cost of fan-out at scale.** Each pipeline start dispatches up to 100 tasks via Redis. At K=500, Redis connection pool exhaustion is the bottleneck — not LLM calls themselves. Fan-out coordination overhead dominates before LLM processing does.
- **CPU is the wrong Worker autoscaling metric (M5-4 + M6-3).** Workers are IO-bound; CPU stays near 0% even under heavy load. Queue depth (LLEN celery) is the correct signal. M6-3 confirms memory, not CPU, grows with concurrent run count.

Limitations and Future Work

- Simulated latency (Experiments 1 & 2): asyncio.sleep() has lower variance than real LLM calls. Real provider throttling affects fan-out differently than uniform sleep.
- Single-node Redis: cache.t3.micro is a single contention point. A replicated cluster would distribute connection pool pressure.
- Load test scope: num_videos=2 is not representative of a full production run. A full-pipeline load test would reveal the interaction between queue depth and Redis pool bottleneck.
- Threshold tuning: 95% was set heuristically. Dynamic tuning based on per-stage observed tail latency would be more robust.

Experiment files:

```
backend/tests/experiments/test_fanout_parallelism.py   — Experiment 1 (5 tests)
backend/tests/experiments/test_straggler_mitigation.py — Experiment 2 (5 tests)
backend/tests/experiments/locustfile.py + run_load_test.sh — Experiment 3 (AWS)
backend/tests/experiments/test_backpressure.py / test_failure_recovery.py /
test_cache_concurrency.py — M5
backend/tests/experiments/test_elasticache_integration.py — M6
```